Operating Systems Lab Assignment: Synchronization and Scheduling

Donovan Reed

October 2026

1 Introduction

This report documents the implementation and analysis of synchronization and scheduling exercises using POSIX threads, mutexes, and condition variables. The programs demonstrate how synchronization prevents race conditions, controls execution order, avoids deadlocks, handles priority issues, and coordinates multiple threads safely.

2 Exercise 1: Hello World

```c
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t lock;
pthread_cond_t cv;
int hello = 0;

void* print_hello(void* arg) {
    pthread_mutex_lock(&lock);

    hello += 1;
    printf("First␣line␣(hello=%d)\n", hello);

    pthread_cond_signal(&cv);
    pthread_mutex_unlock(&lock);

    return NULL;
}

int main() {
    pthread_t thread;

    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&cv, NULL);

    pthread_create(&thread, NULL, print_hello, NULL);

    pthread_mutex_lock(&lock);

    while (hello < 1) {
        pthread_cond_wait(&cv, &lock);
    }

    printf("Second␣line␣(hello=%d)\n", hello);

    pthread_mutex_unlock(&lock);

    pthread_join(thread, NULL);
```

```
    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cv);

    return 0;
}
```

Explanation:

The original program can fail if the shared variable and condition variable are not synchronized correctly. The child thread locks the mutex, increments `hello`, prints the first line, and signals the condition variable. The main thread waits inside a `while` loop until `hello` becomes 1 before printing the second line.

Analysis:

The mutex prevents both threads from modifying or checking `hello` at the same time. The condition variable allows the main thread to sleep until the child thread changes the shared state. The `while` loop is important because a condition variable wakeup does not guarantee that the condition is true until it is checked again.

Screenshot:

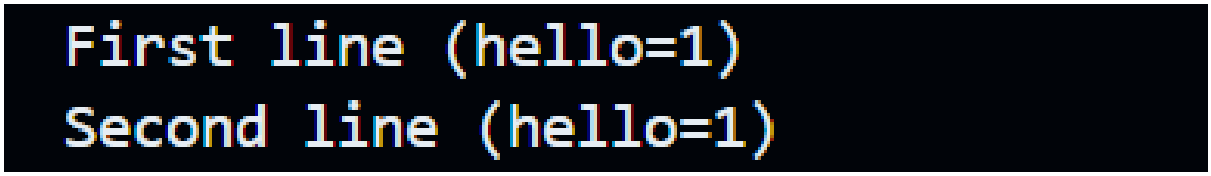


Figure 1: Compilation and execution of hello_world.c

3 Exercise 2: SpaceX Problems

```c
#include <stdio.h>
#include <pthread.h>

pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;

int n = 3;

void* counter(void* arg) {
    while (1) {
        pthread_mutex_lock(&lock);

        if (n <= 0) {
            pthread_cond_broadcast(&cv);
            pthread_mutex_unlock(&lock);
            break;
        }

        printf("%d\n", n);
        n--;

        if (n == 0) {
            pthread_cond_broadcast(&cv);
        }

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}
```

```c
void* announcer(void* arg) {
    pthread_mutex_lock(&lock);

    while (n != 0) {
        pthread_cond_wait(&cv, &lock);
    }

    printf("FALCON␣HEAVY␣TOUCH␣DOWN!\n");

    pthread_mutex_unlock(&lock);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    pthread_create(&t1, NULL, counter, NULL);
    pthread_create(&t2, NULL, announcer, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cv);

    return 0;
}
```

Explanation:

The counter thread prints the values 3, 2, and 1 while protecting the shared variable `n` with a mutex. The announcer thread waits on the condition variable until the countdown reaches zero. It then prints "FALCON HEAVY TOUCH DOWN!".

Analysis:

Without synchronization, the announcer could inspect `n` at the wrong time and print before the countdown completes. The mutex protects the shared counter and the condition variable allows the announcer to wait efficiently. The `while` condition guarantees that the announcement occurs only after `n` reaches zero.

Screenshot:

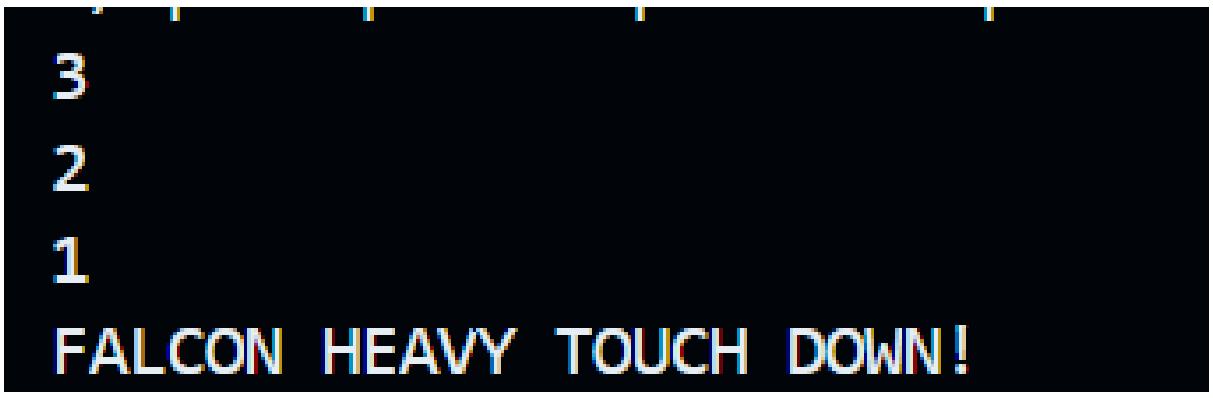


Figure 2: Compilation and execution of spacex.c

4 Exercise 3: I Love You, Unconditionally!

```c
#include <stdio.h>
#include <pthread.h>
```

```c
pthread_mutex_t lock;
pthread_cond_t cv;

int subaru = 0;

void* helper(void* arg) {
    pthread_mutex_lock(&lock);

    subaru += 1;

    pthread_cond_signal(&cv);
    pthread_mutex_unlock(&lock);

    return NULL;
}

int main() {
    pthread_t thread;

    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&cv, NULL);

    pthread_create(&thread, NULL, helper, NULL);

    pthread_mutex_lock(&lock);

    while (subaru != 1) {
        pthread_cond_wait(&cv, &lock);
    }

    if (subaru == 1) {
        printf("I␣love␣Emilia!\n");
    } else {
        printf("I␣love␣Rem!\n");
    }

    pthread_mutex_unlock(&lock);

    pthread_join(thread, NULL);

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cv);

    return 0;
}
```

Explanation:

The helper thread obtains the mutex and increments `subaru`. It then signals the condition variable. The main thread waits until `subaru` equals 1 before printing "I love Emilia!".

Analysis:

The solution follows Mesa semantics because the main thread checks the condition again after waking. The condition variable does not transfer execution directly to the waiting thread, so the shared condition must always be tested in a loop. The mutex also prevents a race condition on `subaru`.

Screenshot:

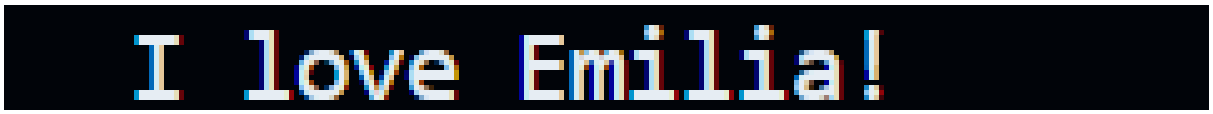


Figure 3: Compilation and execution of love.c

5 Exercise 4: Locking Up the Floopies

```c
#include <stdio.h>
#include <pthread.h>

typedef struct account_t {
    pthread_mutex_t lock;
    float balance;
    long uuid;
} account_t;

typedef struct {
    account_t *donor;
    account_t *recipient;
    float amount;
} transfer_args;

void transfer(account_t *donor, account_t *recipient, float amount) {
    account_t *first =
        (donor->uuid < recipient->uuid) ? donor : recipient;
    account_t *second =
        (donor->uuid < recipient->uuid) ? recipient : donor;

    pthread_mutex_lock(&first->lock);
    pthread_mutex_lock(&second->lock);

    if (donor->balance < amount) {
        printf("Insufficient␣funds.\n");
    } else {
        donor->balance -= amount;
        recipient->balance += amount;

        printf(
            "Transferred␣%.2f␣from␣account␣%ld␣to␣%ld\n",
            amount,
            donor->uuid,
            recipient->uuid
        );
    }

    pthread_mutex_unlock(&second->lock);
    pthread_mutex_unlock(&first->lock);
}

void* transfer_thread(void* arg) {
    transfer_args *args = (transfer_args*)arg;

    transfer(
        args->donor,
        args->recipient,
        args->amount
    );

    return NULL;
}

int main() {
    account_t acc1 = {
        PTHREAD_MUTEX_INITIALIZER,
        1000,
        1
    };
```

```
    account_t acc2 = {
        PTHREAD_MUTEX_INITIALIZER,
        500,
        2
    };

    transfer_args args1 = {
        &acc1,
        &acc2,
        200
    };

    transfer_args args2 = {
        &acc2,
        &acc1,
        100
    };

    pthread_t t1, t2;

    pthread_create(
        &t1,
        NULL,
        transfer_thread,
        &args1
    );

    pthread_create(
        &t2,
        NULL,
        transfer_thread,
        &args2
    );

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final balance account 1: %.2f\n", acc1.balance);
    printf("Final balance account 2: %.2f\n", acc2.balance);

    pthread_mutex_destroy(&acc1.lock);
    pthread_mutex_destroy(&acc2.lock);

    return 0;
}
```

Explanation:

The transfer operation must acquire two account locks. If two threads lock the same accounts in opposite orders, each thread could hold one lock while waiting for the other. This creates a circular wait and can cause deadlock. The corrected program always locks accounts in increasing UUID order.

Analysis:

For example, one thread may transfer from account 1 to account 2 while another transfers from account 2 to account 1. Without a consistent order, one thread could lock account 1 and the other could lock account 2. Both would then wait forever. Ordering the locks by UUID ensures that every thread requests the locks in the same sequence, eliminating circular wait.

Screenshot:

6 Exercise 5: Baking with Condition Variables

```
#include <stdio.h>
#include <pthread.h>
```

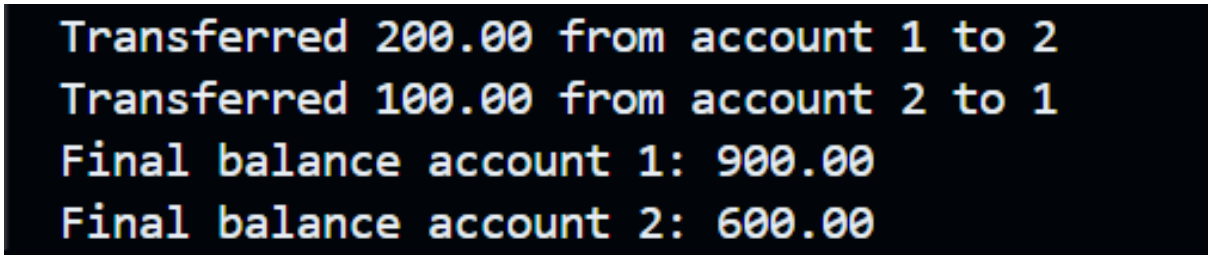


Figure 4: Compilation and execution of floopy.c

```c
#include <stdbool.h>

int numBatterInBowl = 0;
int numEggInBowl = 0;
bool readyToEat = false;

pthread_mutex_t lock;
pthread_cond_t needIngredients;
pthread_cond_t readyToBake;
pthread_cond_t startEating;

void addBatter() {
    numBatterInBowl++;
    printf("Batter added to bowl.\n");
}

void addEgg() {
    numEggInBowl++;
    printf("Egg added to bowl. Eggs: %d\n", numEggInBowl);
}

void heatBowl() {
    printf("Cake is baking...\n");

    readyToEat = true;
    numBatterInBowl = 0;
    numEggInBowl = 0;
}

void eatCake() {
    printf("Cake is ready and being eaten!\n");
    readyToEat = false;
}

void* batterAdder(void* arg) {
    pthread_mutex_lock(&lock);

    while (numBatterInBowl >= 1 || readyToEat) {
        pthread_cond_wait(
            &needIngredients,
            &lock
        );
    }

    addBatter();

    pthread_cond_broadcast(&readyToBake);

    pthread_mutex_unlock(&lock);

    return NULL;
```

```
}

void* eggBreaker(void* arg) {
    pthread_mutex_lock(&lock);

    while (readyToEat) {
        pthread_cond_wait(
            &needIngredients,
            &lock
        );
    }

    while (numEggInBowl < 2) {
        addEgg();
    }

    pthread_cond_broadcast(&readyToBake);

    pthread_mutex_unlock(&lock);

    return NULL;
}

void* bowlHeater(void* arg) {
    pthread_mutex_lock(&lock);

    while (
        numBatterInBowl < 1 ||
        numEggInBowl < 2 ||
        readyToEat
    ) {
        pthread_cond_wait(
            &readyToBake,
            &lock
        );
    }

    heatBowl();

    pthread_cond_signal(&startEating);

    pthread_mutex_unlock(&lock);

    return NULL;
}

void* cakeEater(void* arg) {
    pthread_mutex_lock(&lock);

    while (!readyToEat) {
        pthread_cond_wait(
            &startEating,
            &lock
        );
    }

    eatCake();

    pthread_cond_broadcast(&needIngredients);

    pthread_mutex_unlock(&lock);

    return NULL;
```

```
}

int main() {
    pthread_t batter;
    pthread_t egg;
    pthread_t heater;
    pthread_t eater;

    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&needIngredients, NULL);
    pthread_cond_init(&readyToBake, NULL);
    pthread_cond_init(&startEating, NULL);

    pthread_create(
        &batter,
        NULL,
        batterAdder,
        NULL
    );

    pthread_create(
        &egg,
        NULL,
        eggBreaker,
        NULL
    );

    pthread_create(
        &heater,
        NULL,
        bowlHeater,
        NULL
    );

    pthread_create(
        &eater,
        NULL,
        cakeEater,
        NULL
    );

    pthread_join(batter, NULL);
    pthread_join(egg, NULL);
    pthread_join(heater, NULL);
    pthread_join(eater, NULL);

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&needIngredients);
    pthread_cond_destroy(&readyToBake);
    pthread_cond_destroy(&startEating);

    return 0;
}
```

Explanation:

The baking program coordinates batter addition, egg addition, baking, and eating using a mutex and three condition variables. The batter and egg threads prepare the required ingredients. The heater waits until the bowl has enough batter and eggs, and the eater waits until the cake is ready.

Analysis:

The mutex protects the shared ingredient counts and the `readyToEat` flag. The `readyToBake` condition prevents baking from starting too early. The `startEating` condition prevents the eater from running before baking is complete. The `needIngredients` condition allows another baking cycle to begin only after the previous cake has been eaten.

Screenshot:

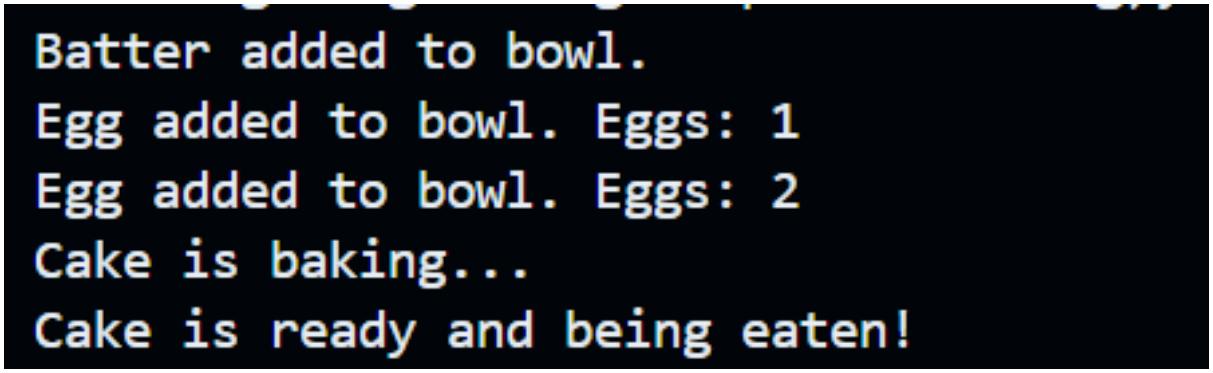


Figure 5: Compilation and execution of baking.c

7 Exercise 6: Priority Donation in Transfer

```c
#include <stdio.h>
#include <pthread.h>

typedef struct account_t {
    pthread_mutex_t lock;
    float balance;
    long uuid;
    int priority;
} account_t;

typedef struct {
    account_t *donor;
    account_t *recipient;
    float amount;
    int priority;
} transfer_args;

void transfer(
    account_t *donor,
    account_t *recipient,
    float amount,
    int thread_priority
) {
    account_t *first =
        (donor->uuid < recipient->uuid)
        ? donor
        : recipient;

    account_t *second =
        (donor->uuid < recipient->uuid)
        ? recipient
        : donor;

    pthread_mutex_lock(&first->lock);

    if (first->priority < thread_priority) {
        printf(
            "Priority donated to account %ld: %d\n",
            first->uuid,
            thread_priority
        );
```

```
        first->priority = thread_priority;
    }

    pthread_mutex_lock(&second->lock);

    if (second->priority < thread_priority) {
        printf(
            "Priority␣donated␣to␣account␣%ld:␣%d\n",
            second->uuid,
            thread_priority
        );

        second->priority = thread_priority;
    }

    if (donor->balance < amount) {
        printf("Insufficient␣funds.\n");
    } else {
        donor->balance -= amount;
        recipient->balance += amount;

        printf(
            "Priority␣%d␣transferred␣%.2f␣from␣account␣%ld␣to␣%ld\n",
            thread_priority,
            amount,
            donor->uuid,
            recipient->uuid
        );
    }

    second->priority = 0;
    first->priority = 0;

    pthread_mutex_unlock(&second->lock);
    pthread_mutex_unlock(&first->lock);
}

void* transfer_thread(void* arg) {
    transfer_args *params =
        (transfer_args*)arg;

    transfer(
        params->donor,
        params->recipient,
        params->amount,
        params->priority
    );

    return NULL;
}

int main() {
    account_t acc1 = {
        PTHREAD_MUTEX_INITIALIZER,
        1000,
        1,
        0
    };

    account_t acc2 = {
        PTHREAD_MUTEX_INITIALIZER,
        500,
```

```
        2,
        0
    };

    transfer_args high = {
        &acc1,
        &acc2,
        200,
        2
    };

    transfer_args low = {
        &acc2,
        &acc1,
        100,
        1
    };

    pthread_t t1, t2;

    pthread_create(
        &t1,
        NULL,
        transfer_thread,
        &high
    );

    pthread_create(
        &t2,
        NULL,
        transfer_thread,
        &low
    );

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf(
        "Final balances: %.2f and %.2f\n",
        acc1.balance,
        acc2.balance
    );

    return 0;
}
```

Explanation:

This program simulates priority donation using the custom priority field allowed by the assignment. When a higher-priority transfer uses an account that has a lower stored priority, the higher priority is temporarily assigned to the account state while the transfer is performed.

Analysis:

Priority inversion occurs when a high-priority thread must wait for a resource held by a lower-priority thread while another medium-priority thread continues running. Priority donation temporarily raises the effective priority associated with the required resource so that the blocking operation can complete sooner. In this implementation, donation is simulated with the custom priority field rather than modifying the Linux scheduler's actual thread priority.

Screenshot:

8 Exercise 7: Barrier Synchronization

```
#include <stdio.h>
```

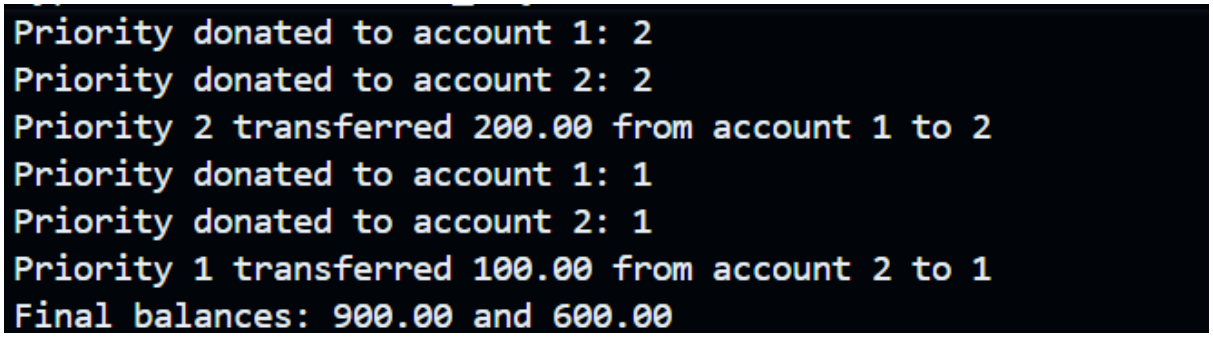


Figure 6: Compilation and execution of priority_transfer.c

```c
#include <pthread.h>
#include <unistd.h>

#define NUM_THREADS 4

pthread_mutex_t lock;
pthread_cond_t cv;

int count = 0;

void barrier() {
    pthread_mutex_lock(&lock);

    count++;

    if (count < NUM_THREADS) {
        while (count < NUM_THREADS) {
            pthread_cond_wait(&cv, &lock);
        }
    } else {
        pthread_cond_broadcast(&cv);
    }

    pthread_mutex_unlock(&lock);
}

void* worker(void* arg) {
    int id = *(int*)arg;

    printf("Thread␣%d:␣Before␣barrier\n", id);

    sleep(id % 2);

    barrier();

    printf("Thread␣%d:␣After␣barrier\n", id);

    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    int ids[NUM_THREADS];

    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&cv, NULL);

    for (int i = 0; i < NUM_THREADS; i++) {
```

```
        ids[i] = i;

        pthread_create(
            &threads[i],
            NULL,
            worker,
            &ids[i]
        );
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&cv);

    return 0;
}
```

Explanation:

The barrier uses a mutex, condition variable, and shared counter. Each worker increments the counter when it reaches the barrier. Threads arriving before the final thread wait on the condition variable. When the final thread arrives, it broadcasts to wake all waiting threads.

Analysis:

The barrier ensures that no thread prints its "After barrier" message until every thread has reached the synchronization point. The mutex protects the shared counter, and the condition variable prevents waiting threads from continuing too early.

Screenshot:

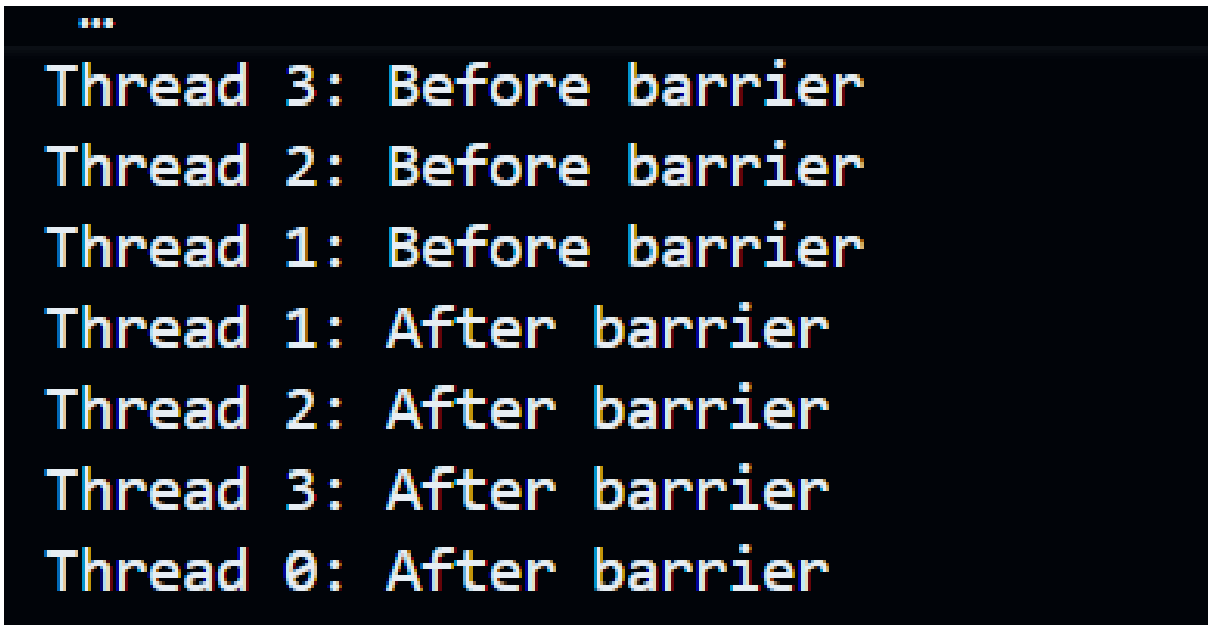


Figure 7: Compilation and execution of barrier.c

9 Exercise 8: Readers-Writers with Priority

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```
pthread_mutex_t lock;
pthread_cond_t reader_cv;
pthread_cond_t writer_cv;

int reader_count = 0;
int writer_waiting = 0;
int writer_active = 0;
int shared_data = 0;

void* reader(void* arg) {
    int id = *(int*)arg;

    pthread_mutex_lock(&lock);

    while (writer_waiting > 0 || writer_active) {
        pthread_cond_wait(&reader_cv, &lock);
    }

    reader_count++;

    pthread_mutex_unlock(&lock);

    printf("Reader␣%d␣reads:␣%d\n", id, shared_data);

    usleep(100000);

    pthread_mutex_lock(&lock);

    reader_count--;

    if (reader_count == 0) {
        pthread_cond_signal(&writer_cv);
    }

    pthread_mutex_unlock(&lock);

    return NULL;
}

void* writer(void* arg) {
    int id = *(int*)arg;

    pthread_mutex_lock(&lock);

    writer_waiting++;

    while (reader_count > 0 || writer_active) {
        pthread_cond_wait(&writer_cv, &lock);
    }

    writer_waiting--;
    writer_active = 1;

    shared_data++;

    printf("Writer␣%d␣writes:␣%d\n", id, shared_data);

    writer_active = 0;

    if (writer_waiting > 0) {
        pthread_cond_signal(&writer_cv);
    } else {
        pthread_cond_broadcast(&reader_cv);
```

```
    }

    pthread_mutex_unlock(&lock);

    return NULL;
}

int main() {
    pthread_t readers[3];
    pthread_t writers[2];

    int reader_ids[3] = {1, 2, 3};
    int writer_ids[2] = {1, 2};

    pthread_mutex_init(&lock, NULL);
    pthread_cond_init(&reader_cv, NULL);
    pthread_cond_init(&writer_cv, NULL);

    for (int i = 0; i < 3; i++) {
        pthread_create(
            &readers[i],
            NULL,
            reader,
            &reader_ids[i]
        );
    }

    usleep(50000);

    for (int i = 0; i < 2; i++) {
        pthread_create(
            &writers[i],
            NULL,
            writer,
            &writer_ids[i]
        );
    }

    for (int i = 0; i < 3; i++) {
        pthread_join(readers[i], NULL);
    }

    for (int i = 0; i < 2; i++) {
        pthread_join(writers[i], NULL);
    }

    pthread_mutex_destroy(&lock);
    pthread_cond_destroy(&reader_cv);
    pthread_cond_destroy(&writer_cv);

    return 0;
}
```

Explanation:

The program allows multiple readers to access shared data simultaneously while giving waiting writers priority. A reader waits whenever a writer is active or when `writer_waiting` is greater than zero. A writer waits until no readers or other writer are active.

Analysis:

Writer priority is enforced by preventing new readers from entering once a writer begins waiting. Existing readers are allowed to finish, and then the writer receives access. After a writer completes, another waiting writer is signaled first. If no writers remain, all waiting readers are released.

Screenshot:

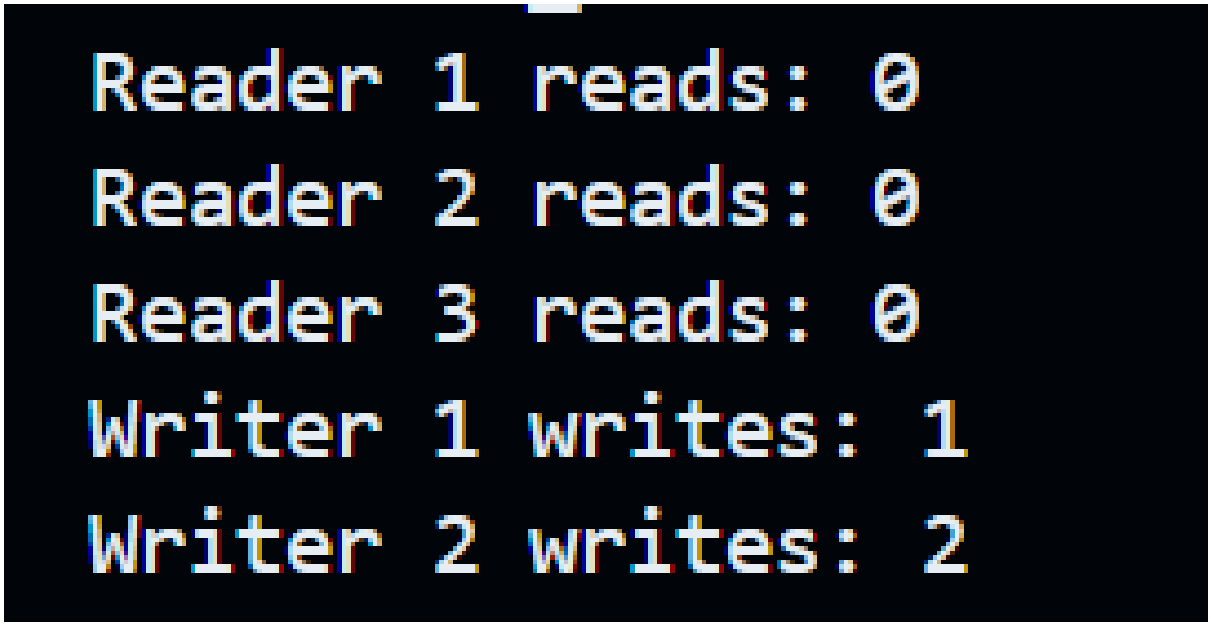


Figure 8: Compilation and execution of readers_writers.c

10 Exercise 9: Thread Pool

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <stdbool.h>

#define NUM_THREADS 4
#define NUM_TASKS 10

typedef struct {
    void (*task)(int);
    int arg;
} Task;

typedef struct {
    Task *queue;
    int head;
    int tail;
    int count;
    int size;
    bool shutdown;

    pthread_mutex_t lock;
    pthread_cond_t not_empty;
    pthread_cond_t not_full;
} ThreadSafeQueue;

void queue_init(ThreadSafeQueue *q, int size) {
    q->queue = malloc(sizeof(Task) * size);
    q->head = 0;
    q->tail = 0;
    q->count = 0;
    q->size = size;
    q->shutdown = false;

    pthread_mutex_init(&q->lock, NULL);
    pthread_cond_init(&q->not_empty, NULL);
```

```
    pthread_cond_init(&q->not_full, NULL);
}

void queue_push(ThreadSafeQueue *q, Task task) {
    pthread_mutex_lock(&q->lock);

    while (q->count == q->size) {
        pthread_cond_wait(&q->not_full, &q->lock);
    }

    q->queue[q->tail] = task;
    q->tail = (q->tail + 1) % q->size;
    q->count++;

    pthread_cond_signal(&q->not_empty);

    pthread_mutex_unlock(&q->lock);
}

int queue_pop(ThreadSafeQueue *q, Task *task) {
    pthread_mutex_lock(&q->lock);

    while (q->count == 0 && !q->shutdown) {
        pthread_cond_wait(&q->not_empty, &q->lock);
    }

    if (q->count == 0 && q->shutdown) {
        pthread_mutex_unlock(&q->lock);
        return 0;
    }

    *task = q->queue[q->head];

    q->head = (q->head + 1) % q->size;
    q->count--;

    pthread_cond_signal(&q->not_full);

    pthread_mutex_unlock(&q->lock);

    return 1;
}

void* worker(void* arg) {
    ThreadSafeQueue *q = (ThreadSafeQueue*)arg;

    while (1) {
        Task task;

        if (!queue_pop(q, &task)) {
            break;
        }

        task.task(task.arg);
    }

    return NULL;
}

void sample_task(int arg) {
    printf(
        "Task executed with arg: %d by thread %lu\n",
        arg,
```

```
        (unsigned long)pthread_self()
    );
}

int main() {
    ThreadSafeQueue q;

    queue_init(&q, NUM_TASKS);

    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(
            &threads[i],
            NULL,
            worker,
            &q
        );
    }

    for (int i = 0; i < NUM_TASKS; i++) {
        Task task = {
            sample_task,
            i
        };

        queue_push(&q, task);
    }

    pthread_mutex_lock(&q.lock);

    while (q.count > 0) {
        pthread_mutex_unlock(&q.lock);
        sched_yield();
        pthread_mutex_lock(&q.lock);
    }

    q.shutdown = true;

    pthread_cond_broadcast(&q.not_empty);

    pthread_mutex_unlock(&q.lock);

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    pthread_mutex_destroy(&q.lock);
    pthread_cond_destroy(&q.not_empty);
    pthread_cond_destroy(&q.not_full);

    free(q.queue);

    return 0;
}
```

Explanation:

The thread pool creates a fixed number of worker threads and stores tasks inside a thread-safe queue. Workers wait for tasks using a condition variable and remove available tasks from the queue. The mutex protects the queue's head, tail, count, and shutdown state.

Analysis:

A thread pool is more efficient than creating a new thread for every task because the worker threads are created once and reused. This reduces repeated thread creation and destruction overhead. The

`not_empty` and `not_full` condition variables coordinate producers and workers while preventing race conditions in the queue.

Screenshot:

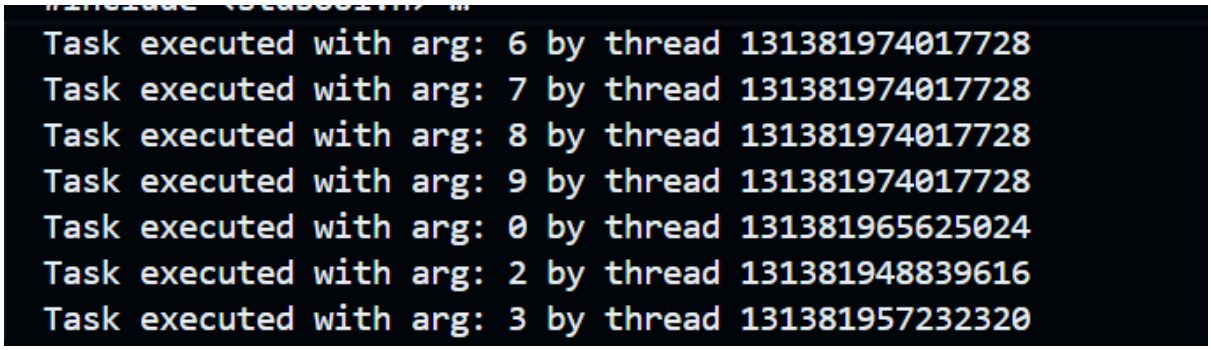


Figure 9: Compilation and execution of thread_pool.c

11 Conclusion

These exercises demonstrate how POSIX synchronization mechanisms can coordinate concurrent programs safely. Mutexes provide mutual exclusion for shared state, while condition variables allow threads to wait for specific conditions without continuously polling. Consistent lock ordering prevents deadlocks, writer-priority rules control access order, barriers synchronize groups of threads, and thread pools efficiently reuse worker threads.